CEFET/RJ - CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA

Unificação de Sistemas e seus impactos: Análise Teórica sobre Benefícios e Análise sobre a futura implementação do aplicativo InLight

Mauro Felippe Telles Junior

Prof. Orientador:
Cristiano Fuschilo

Rio de Janeiro,
Novembro de 2025

CEFET/RJ - CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA

Unificação de Sistemas e seus impactos: Análise Teórica sobre Benefícios e Análise sobre a futura implementação do aplicativo InLight

Mauro Felippe Telles Junior

Projeto final apresentado em cumprimento às normas do Departamento de Educação Superior do Centro Federal de Educação Tecnológica Celso Suckow da Fonseca, CEFET/RJ, como parte dos requisitos para obtenção do título de Bacharel em Sistemas de Informação.

Prof. Orientador:
Cristiano Fuschilo

Rio de Janeiro,
Novembro de 2025

DEDICATÓRIA

Texto

AGRADECIMENTOS

Texto

“Só seria fácil se não fosse difícil’

(Nome do autor)

RESUMO

Texto

Palavras-chaves: Palavra; Palavra; Palavra; Palavra; Palavra

ABSTRACT

Texto

Keywords: Palavra; Palavra; Palavra; Palavra; Palavra

Sumário

1 Introdução **2**

1.1 Contextualização 2

1.2 Problema Investigado 3

1.3 Motivação 3

1.4 Objetivos 3

1.5 Justificativa 4

1.6 Questões de Pesquisa para o Estudo de Caso 4

1.7 Delimitação do Estudo 4

1.8 Estrutura do Estudo 5

2 Fundamentação Teórica **6**

2.1 InLight 6

2.2 Tecnologias 6

2.2.1 Arquitetura da Aplicação 7

2.2.2 Front-end: React Native 7

2.2.3 Back-end: NestJS e Node.js 8

2.2.4 Banco de Dados e Persistência 9

2.2.5 Autenticação e Segurança 10

2.2.6 Validação e Tratamento de Dados 12

2.2.7 Interceptors e Exception Filters 13

2.2.8 Processamento de Arquivos Excel 13

2.2.9 Comunicação HTTP e CORS 14

2.2.10 Testes Automatizados 14

2.2.11 Considerações sobre a Stack Tecnológica 15

2.3 Sistemas de Informação e Integração 15

2.3.1 Conceitos Fundamentais de Sistemas de Informação 15

2.3.2 Sistemas Legados e suas Limitações 16

2.3.3 Fragmentação de Sistemas e seus Impactos 17

2.3.4 Integração de Sistemas: Conceitos e Abordagens 18

2.3.5 Unificação de Sistemas: Conceito e Benefícios 19

2.3.6 Desafios na Unificação de Sistemas 20
2.4 Gestão de Processos e Workflows 21
2.4.1 Conceitos de Processos de Negócio 21
2.4.2 Modelagem e Análise de Processos 22
2.4.3 Reengenharia de Processos 22
2.4.4 Automação de Processos 23
2.4.5 Workflows e Gestão de Tarefas 24
2.4.6 Redução de Retrabalho 25
2.5 Transformação Digital no Setor Elétrico 26
2.5.1 Contexto do Setor Elétrico Brasileiro 26
2.5.2 CCEE - Câmara de Comercialização de Energia Elétrica 26
2.5.3 ACR e ACL: Ambientes de Contratação 27
2.5.4 Desafios de Gestão de Contratos 28
2.5.5 Obrigações e Prazos Regulatórios 28
2.5.6 Transformação Digital no Setor 29
2.5.7 Integração de Sistemas no Contexto Regulatório 30
2.5.8 Casos de Sucesso e Benchmarks 30

3 Revisão da Literatura / Trabalhos Relacionados 31

4 Proposta 32
4.1 InLight 32
4.2 Tecnologias 32
4.2.1 Arquitetura da Aplicação 33
4.2.2 Front-end: React Native 33
4.2.3 Back-end: NestJS e Node.js 34
4.2.4 Banco de Dados e Persistência 35
4.2.5 Autenticação e Segurança 36
4.2.6 Validação e Tratamento de Dados 38
4.2.7 Interceptors e Exception Filters 39
4.2.8 Processamento de Arquivos Excel 39
4.2.9 Comunicação HTTP e CORS 40
4.2.10 Testes Automatizados 40

4.2.11 Considerações sobre a Stack Tecnológica 41
4.3 Sistemas de Informação e Integração 41
4.3.1 Conceitos Fundamentais de Sistemas de Informação 41
4.3.2 Sistemas Legados e suas Limitações 42
4.3.3 Fragmentação de Sistemas e seus Impactos 43
4.3.4 Integração de Sistemas: Conceitos e Abordagens 44
4.3.5 Unificação de Sistemas: Conceito e Benefícios 45
4.3.6 Desafios na Unificação de Sistemas 46
4.4 Gestão de Processos e Workflows 47
4.4.1 Conceitos de Processos de Negócio 47
4.4.2 Modelagem e Análise de Processos 48
4.4.3 Reengenharia de Processos 48
4.4.4 Automação de Processos 49
4.4.5 Workflows e Gestão de Tarefas 50
4.4.6 Redução de Retrabalho 51
4.5 Transformação Digital no Setor Elétrico 52
4.5.1 Contexto do Setor Elétrico Brasileiro 52
4.5.2 CCEE - Câmara de Comercialização de Energia Elétrica 52
4.5.3 ACR e ACL: Ambientes de Contratação 53
4.5.4 Desafios de Gestão de Contratos 54
4.5.5 Obrigações e Prazos Regulatórios 54
4.5.6 Transformação Digital no Setor 55
4.5.7 Integração de Sistemas no Contexto Regulatório 56
4.5.8 Casos de Sucesso e Benchmarks 56

5 Conclusão **57**

Referências **58**

Lista de Figuras

Lista de Tabelas

Capítulo 1

Introdução

1.1 Contextualização

Atualmente, nas organizações contemporâneas, a informação e a forma como ela é transmitida tornaram-se um dos ativos mais valiosos. Segundo Turban and Volonino [2010], a informação é essencial para apoiar decisões estratégicas e operacionais. A gestão eficiente desse recurso depende, em grande parte, da capacidade dos sistemas corporativos e dos colaboradores em integrar dados, informações, processos e pessoas, reforçando o que afirmam Laudon and Laudon [2020] ao destacar que sistemas fragmentados prejudicam o desempenho organizacional.

Entretanto, muitas empresas ainda operam com sistemas legados e fragmentados, desenvolvidos em épocas em que a estrutura tecnológica e os métodos de integração não eram adequados, o que dificulta a comunicação entre sistemas. Esse cenário resulta em retrabalho, desperdício de tempo, inconsistências e maior dificuldade na tomada de decisão, conforme apontam Stair and Reynolds [2018].

No setor de energia elétrica, como é o caso da empresa Light, a dependência de sistemas integrados é ainda mais evidente. Operações de campo, monitoramento de equipes, gestão de contratos e atendimento ao cliente exigem soluções tecnológicas capazes de consolidar informações em tempo real e oferecer suporte confiável às rotinas críticas, como defendem Rezende [2017].

Especificamente na área de Compra de Energia, podem ser citados os contratos do ACR (Ambiente de Contratação Regulada) e do ACL (Ambiente de Contratação Livre), que dependem de informações provenientes de diferentes sistemas. Um exemplo disso é a CCEE (Câmara de Comercialização de Energia Elétrica), na qual há o ingresso de notas em outro sistema cuja base de dados ainda não é automatizada nem integrada aos demais sistemas corporativos — um problema recorrente em organizações que carecem de integração plena, como discutido por Pressman and Maxim [2016].

1.2 Problema Investigado

Diante desse cenário, identifica-se que, com o avanço dos mercados ACR e ACL, os benefícios e impactos da unificação de sistemas se tornam cada vez mais relevantes, especialmente considerando o aumento do volume de dados e de clientes. Para Davenport [2018], sistemas integrados promovem inteligência operacional e reduzem gargalos informacionais. Assim, soluções unificadas, como o aplicativo *InLight*, tornam-se essenciais para otimizar processos, reduzir erros e centralizar informações estratégicas.

1.3 Motivação

A escolha do tema está relacionada ao atual cenário de transformação digital, no qual a integração e a modernização de sistemas são estratégias fundamentais para ganhos de eficiência. Segundo Schwab [2016], a digitalização intensifica a necessidade de automação e integração de processos. No caso da Light, que abrange diversas áreas e sistemas corporativos, a comunicação eficaz e a padronização das informações são fatores críticos para o bom desempenho das operações Laudon and Laudon [2020].

Assim, este estudo se justifica tanto como contribuição prática à organização quanto como exercício acadêmico de aplicação dos conhecimentos adquiridos no curso de Sistemas de Informação..

1.4 Objetivos

O objetivo geral deste trabalho é analisar os benefícios e impactos da unificação de sistemas corporativos, a partir de uma abordagem teórica e de uma análise da possível da implementação do aplicativo *InLight*.

Objetivos específicos:

- Mapear os principais desafios e limitações decorrentes da fragmentação de sistemas;
- Revisar literatura acadêmica e técnica sobre unificação tecnológica e seus efeitos organizacionais;
- Descrever como seria o processo de implementação do aplicativo *InLight* na empresa Light junto com seus impactos;

- Mensurar os possíveis resultados que podem ser obtidos após a adoção do aplicativo, considerando indicadores de eficiência, integração e usabilidade;
- Identificar oportunidades de melhoria e perspectivas futuras para sistemas unificados em ambientes corporativos.

1.5 Justificativa

A unificação de sistemas corporativos representa um passo fundamental para organizações que buscam maior eficiência, confiabilidade e segurança na gestão da informação. Segundo Rezende [2017], a integração tecnológica reduz redundâncias, melhora o fluxo de informações e fortalece a governança organizacional. No contexto da Light, tal integração possibilita o aperfeiçoamento dos fluxos de trabalho, a redução de retrabalho e a melhoria de processos críticos.

Portanto, o estudo contribui tanto para o aprimoramento das práticas empresariais quanto para o avanço das discussões acadêmicas sobre sistemas integrados e transformação digital Davenport [2018].

1.6 Questões de Pesquisa para o Estudo de Caso

A partir do problema apresentado, este Trabalho de Conclusão de Curso busca responder às seguintes questões:

- Como sistemas fragmentados impactam a eficiência operacional da empresa?
- De que forma a unificação de sistemas, por meio do aplicativo InLight, contribui para a integração de processos internos e a melhoria do fluxo de informações?
- Como observar e mensurar os benefícios e limitações decorrentes da unificação de sistemas no ambiente corporativo?
- Como a visão dos usuários e gestores influencia a adoção de sistemas unificados?

1.7 Delimitação do Estudo

Este trabalho visa à análise teórica e prática da unificação de sistemas corporativos, com foco na implementação de sistemas, adequação a novas tecnologias e seus impactos. Como

orienta Yin [2015], estudos de caso permitem investigar fenômenos contemporâneos dentro de seu contexto real, justificando a abordagem adotada.

A coleta de dados será realizada com base em documentos institucionais disponibilizados, sem acesso a informações confidenciais, preservando o sigilo e a ética profissional, conforme recomenda Gil [2019].

1.8 Estrutura do Estudo

O trabalho está estruturado em seis capítulos, conforme descrito a seguir:

- **Capítulo 1 - Introdução:** apresenta a contextualização do tema, problema investigado, motivação, objetivos, justificativa, questões de pesquisa, delimitação do estudo e estrutura geral;
- **Capítulo 2 - Fundamentação Teórica:** reúne conceitos e definições sobre sistemas de informação, unificação com novas tecnologias, integração de processos e impactos dentro de uma organização;
- **Capítulo 3 - Revisão da Literatura:** sistematiza pesquisas acadêmicas e técnicas que abordam o tema em questão, destacando autores e estudos relevantes para o embasamento teórico, trazendo pontos positivos e negativos sobre a unificação e suas aplicações;
- **Capítulo 4 - Estudo de Caso: Aplicativo InLight:** descreve o processo de concepção, desenvolvimento e implementação do aplicativo na empresa Light, além de como vem modificando o dia a dia da empresa;
- **Capítulo 5 - Análise dos Resultados:** apresenta a comparação entre os cenários pré e pós-implementação, destacando ganhos de eficiência, integração e percepção dos usuários;
- **Capítulo 6 - Conclusões e Recomendações:** desfecho final do nosso estudo, discutindo o contexto geral abordado além de sugestões futuras tanto no contexto Light quanto sobre unificação de sistemas.

Capítulo 2

Fundamentação Teórica

Este capítulo tem como objetivo apresentar os conceitos, tencologias e práticas utilizadas no projeto. Irá ser abordado temas que se relacionam com o aplicativo InLight, com suas respectivas tecnologias, pontos positivos e negativos em práticas na implementação de unificação de sistemas e como deve ser realizado.

2.1 InLight

O InLight é um aplicativo de unificação de sistemas, desenvolvido em TypeScript, que oferece diversas aplicações, visto que agrupa as principais informações dos principais sistemas utilizados não somente no BackOffice ACR, mas como na LIGHT como um todo. É uma aplicação especifica para o mercado regulado podendo ser ampliado para o mercado livre.

Criado e desenvolvido para tratar informações da CCEE, Thunders e informações que só eram armazenadas em planilhas.

Ele foi estruturado em 3 principais módulos:

- Notas - agrupando contratos de quantidade, disponibilidade e cotas e todos os seus blocos, além de um contador na tela inicial para o próximo bloco.
- Metas - de acordo com a nova demanda, se foi necessário criar uma tela para metas de uma forma bem dinâmica.
- Obrigações - pagamentos chamados de especiais pelo modo que são tratados em seu pagamento.

2.2 Tecnologias

Abordando o desenvolvimento do aplicativo InLight, foi construído com um base de tecnologias que garantem modernidade e uma maior facilidade em alterações futuras. A partir deste ponto, irá ser apresentado as principais tecnologias utilizadas, organizadas de acordo com as suas funcionalidade e propósitos dentro do aplicativo.

2.2.1 Arquitetura da Aplicação

O InLight foi construído com uma arquitetura cliente-servidor, realizando assim uma boa divisão da interface do usuário (front-end) e a estrutura lógica de negócio (back-end). De acordo com, Pressman and Maxim [2016], realizando essa estrutura irá promover uma maior facilidade na manutenção e na escalabilidade, além de facilitar que equipes diferentes trabalhem no projeto.

2.2.2 Front-end: React Native

React Native e TypeScript

Falando sobre o front-end do InLight, ele foi desenvolvido em React Native, que é um Framework JavaScript mantido pelo Facebook (Meta), esta tecnologia permite o desenvolvimento de aplicações mobile para Android e iOS a partir de uma única base de código Eisenman [2015]. Partindo deste viés, a escolha do React Native se justifica por:

- **Desenvolvimento multiplataforma:** como não é necessário manter duas bases separadamente, reduz consideravelmente o tempo e custo de desenvolvimento
- **Hot Reload:** permite visualizar atualizações no código em tempo real, assim tornando o processo de desenvolvimento mais rápido.
- **Comunidade ativa:** Comunidade bastante engajada e boa diversidade de bibliotecas.

Ademais, o projeto foi desenvolvido em TypeScript, que acrescenta a tipagem estática opcional, já que o JavaScript é uma linguagem de tipagem dinâmica. Seguindo essa filosofia, proporciona uma maior segurança no desenvolvimento e facilitará a manutenção no código como mencionado por Bierman et al. [2014]. Seguindo a análise de, Microsoft [2020], a utilização da tipagem estática reduz a chance de ocorrer erros durante a execução e melhora a experiência de desenvolvimento através autocompletar e refatoração.

Gerenciamento de Estado e Navegação

Para o gerenciamento de estado global, o InLight utiliza a Context API do React, este recurso permite compartilhar dados entre componentes sem necessidade de passar propriedades

manualmente através das árvores de componentes Facebook [2023a]. O contexto de autenticação (AuthContext) é responsável no código por gerenciar informações do usuário logado, tokens de acesso e estado de autenticação. Abordando a troca entre telas, foi utilizado a biblioteca React Navigation, falando mais especificamente foi utilizado o padrão Stack Navigator. Essa biblioteca oferece navegação nativa, transições suaves e gerenciamento de histórico de navegação React Navigation [2023]. Na parte de locação de dados, como por exemplo tokens de autenticação, foi utilizado AsyncStorage, que fornece um sistema de armazenamento chave-valor assíncrono e não criptografado Facebook [2023b].

Componentes e Interface de Usuário

A interface do usuário foi desenvolvida utilizando os componentes básicos do React Native (View, Text, TouchableOpacity, FlatList, ScrollView), complementados por bibliotecas mais específicas:

- **@react-native-community/slider:** componente de controle deslizante para ajuste de progresso de metas, sendo bem dinâmica em todas as telas;
- **@react-native-picker/picker:** seletor de opções para escolha de papéis de usuário;
- **expo-document-picker:** seleção de documentos do dispositivo para funcionalidade de upload, utilizado na parte de pendências.

2.2.3 Back-end: NestJS e Node.js

Framework NestJS

O back-end do InLight é desenvolvido em NestJS, um framework progressivo para construção de aplicações server-side eficientes e escaláveis NestJS [2023]. O NestJS é construído em Node.js e utiliza Typescript como linguagem padrão, garantindo uma boa conexão com o front-end. As principais características do NestJS que justificam sua adoção incluem:

- **Arquitetura modular:** organização do código em módulos independentes e reutilizáveis;
- **Injeção de dependências:** padrão que facilita testes e manutenção do código;
- **Suporte a TypeScript:** aproveitamento o máximo dos benefícios da tipagem estática;

- **Decorators:** metaprogramação que torna o código mais declarativo e legível;
- **Estrutura baseada em Angular:** familiaridade para desenvolvedores que conhecem frameworks modernos.

Arquitetura em Camadas

O back-end segue uma arquitetura em três camadas bem definidas, conforme recomendado por Fowler [2002]:

1. **Camada de Apresentação (Controllers):** responsável por receber requisições HTTP, validar entradas e retornar respostas adequadas. Utiliza em seu código decorators como por exemplo: @Controller, @Get, @Post, @Patch e @Delete para definir rotas e métodos HTTP;
2. **Camada de Negócio (Services):** contém a lógica de negócio do aplicativo, implementando regras, validações e orquestrando operações entre diferentes entidades;
3. **Camada de Dados (Repositories):** responsável pela comunicação com o banco de dados, abstraindo detalhes de persistência através do padrão Repository.

Esta separação promove o princípio da responsabilidade única (Single Responsibility Principle) do SOLID Martin [2008], facilitando manutenção, testes e evolução do sistema.

2.2.4 Banco de Dados e Persistência

PostgreSQL

O InLight utiliza PostgreSQL como sistema gerenciador de banco de dados. Este banco dados é um open-source robusto, conhecido por sua confiabilidade, integridade de dados e conformidade com padrões SQL PostgreSQL Global Development Group [2023]. Segundo Obe and Hsu [2017], PostgreSQL oferece recursos avançados como:

- Suporte completo a transações ACID (Atomicidade, Consistência, Isolamento, Durabilidade);
- Tipos de dados complexos e extensíveis;
- Índices avançados para otimização de consultas;

- Suporte a JSON e dados semi-estruturados;
- Replicação e alta disponibilidade.

TypeORM

Para a comunicação entre a aplicação e o banco de dados, utiliza-se TypeORM, um Object-Relational Mapping (ORM) que permite trabalhar com entidades de banco de dados como objetos TypeScript TypeORM [2023]. O TypeORM oferece:

- **Query Builder:** construção de consultas complexas de forma programática e type-safe;
- **Migrations:** versionamento e controle de mudanças no schema do banco;
- **Relacionamentos:** definição declarativa de relações entre entidades (OneToMany, ManyToOne, ManyToMany);
- **Lazy Loading e Eager Loading:** estratégias de carregamento de relacionamentos.

O uso de ORM reduz significativamente a quantidade de código SQL escrito manualmente, previne vulnerabilidades como SQL Injection e facilita a portabilidade entre diferentes SGBDs Ambler [2003].

2.2.5 Autenticação e Segurança

JSON Web Tokens (JWT)

O sistema de autenticação do InLight baseia-se em JSON Web Tokens (JWT), um padrão aberto (RFC 7519) para transmissão segura de informações entre partes como um objeto JSON Jones et al. [2015]. O fluxo de autenticação funciona da seguinte forma:

1. Usuário envia credenciais (email e senha) para o endpoint de login;
2. Servidor valida as credenciais e, se válidas, gera um token JWT contendo informações do usuário;
3. Cliente armazena o token localmente e o envia no header Authorization de requisições subsequentes;
4. Servidor valida o token em cada requisição protegida.

Os benefícios do JWT incluem:

- **Stateless:** servidor não precisa armazenar sessões, facilitando escalabilidade;
- **Autocontido:** token contém todas as informações necessárias para autenticação;
- **Compacto:** pode ser transmitido via URL, POST ou HTTP header;
- **Seguro:** pode ser assinado digitalmente garantindo integridade.

Passport.js e Estratégias de Autenticação

A implementação da autenticação utiliza Passport.js, um middleware de autenticação flexível e modular para Node.js Jared Hanson [2023]. É aplicado passport-jwt para validação de tokens JWT. O NestJS integra o Passport através do modulo @nestjs/passport, fornecendo Guards que protegem rotas específicas.

Criptografia de Senhas

Para o armazenamento seguro de senhas, utiliza-se a biblioteca bcrypt, que implementa o algoritmo de hashing bcrypt baseado no Blowfish cipher Provos and Mazières [1999]. Características importantes:

- **Hash com salt:** cada senha recebe um salt único, prevenindo ataques de rainbow table;
- **Função de custo adaptável:** permite aumentar a complexidade computacional ao longo do tempo;
- **Resistente a ataques de força bruta:** custo computacional alto, porém dificulta quebrar senhas.

Controle de Acesso Baseado em Papéis (RBAC)

O InLight implementa RBAC (Role-Based Access Control) com três níveis de acesso:

- **Admin:** acesso total ao sistema, incluindo cadastro de usuários e importação de dados;
- **Manager:** acesso intermediário com permissões de gestão;
- **User:** acesso básico às funcionalidades essenciais.

Segundo Ferraiolo et al. [2001], RBAC simplifica a administração de permissões e garante conformidade com políticas de segurança organizacionais.

2.2.6 Validação e Tratamento de Dados

Data Transfer Objects (DTOs)

No aplicativo InLight o padrão DTO é utilizado para transferência de dados entre camadas Fowler [2002]. DTOs são objetos simples que transportam dados, sem conter lógica de negócio, que é principalmente no service. No contexto do projeto, existem DTOs específicos para:

- **CreateDTO:** dados necessários para criação de entidades;
- **UpdateDTO:** dados para atualização (geralmente PartialType do CreateDTO);
- **FilterDTO:** parâmetros de filtro para consultas;
- **ResponseDTO:** formatação de respostas da API.

Class-validator

A validação de dados de entrada é realizada através da biblioteca class-validator, que utiliza decorators TypeScript para definir regras de validação. Exemplos de decorators utilizados:

- @IsNotEmpty: campo obrigatório;
- @IsEmail: validação de formato de email;
- @IsDateString: validação de formato de data;
- @IsNumber: validação de tipo numérico;
- @IsString: validação de tipo string.

ValidationPipe

O NestJS fornece o recurso ValidationPipe, um pipe global que automaticamente valida payloads de requisições nos DTOs definidos. Configurado no arquivo main.ts com as opções:

- **whitelist: true** - remove propriedades não definidas no DTO;
- **forbidNonWhitelisted: true** - rejeita requisições com propriedades extras;
- **transform: true** - converte automaticamente tipos primitivos.

Essa abordagem garante que apenas dados válidos cheguem às camadas de negócio e dados.

2.2.7 Interceptors e Exception Filters

Response Interceptor

O ResponseInterceptor padroniza o formato de todas as respostas da API, encapsulando dados em uma estrutura consistente. Isso facilita o tratamento no front-end e melhora a experiência do desenvolvedor NestJS [2023].

HttpExceptionFilter

O HttpExceptionFilter customizado captura exceções HTTP e formata respostas de erro de maneira uniforme, incluindo:

- Código de status HTTP apropriado;
- Mensagem de erro descritiva;
- Timestamp da ocorrência;
- Path da requisição;
- Detalhes adicionais quando apropriado.

Segundo Pressman and Maxim [2016], tratamento adequado de exceções é fundamental para robustez e manutenibilidade de sistemas.

2.2.8 Processamento de Arquivos Excel

Uma funcionalidade muito importante do InLight é a importação de através de arquivos Excel (.xlsx), especialmente para obrigações futuras. O processo inclui:

- **Upload via FormData:** suporte a multipart/form-data;
- **Conversão de datas seriais:** Excel armazena datas como números seriais desde 1900-01-01, exigindo conversão para formato padrão;
- **Validação de dados:** verificação de formato e integridade antes de persistir;
- **Feedback ao usuário:** notificação de sucesso ou erro detalhado.

2.2.9 Comunicação HTTP e CORS

RESTful API

A API do InLight segue os princípios REST (Representational State Transfer) definidos por Fielding [2000]:

- **Cliente-Servidor:** separação clara de responsabilidades;
- **Stateless:** cada requisição contém todas as informações necessárias;
- **Cacheable:** respostas podem ser cacheadas quando apropriado;
- **Interface uniforme:** uso consistente de métodos HTTP;
- **Sistema em camadas:** arquitetura pode incluir intermediários.

CORS (Cross-Origin Resource Sharing)

O back-end habilita CORS para permitir que o aplicativo mobile acesse a API. A configuração no main.ts especifica:

- Origins permitidas;
- Métodos HTTP aceitos (GET, POST, PATCH, DELETE);
- Suporte a credenciais;

Essa configuração é essencial para comunicação segura entre diferentes origens W3C [2014].

2.2.10 Testes Automatizados

O aplicativo utiliza Jest como framework de testes, permitindo:

- **Testes unitários:** validação de componentes isolados;
- **Testes de integração:** verificação de interação entre módulos;
- **Mocking:** simulação de dependências externas;
- **Coverage reports:** análise de cobertura de código.

O NestJS fornece @nestjs/testing, que facilita a criação de módulos de teste com injeção de dependências NestJS [2023].

2.2.11 Considerações sobre a Stack Tecnológica

A escolha das tecnologias empregadas no InLight reflete uma busca por modernidade, escalabilidade e manutenibilidade. O uso de TypeScript em toda a sua estrutura garante consistência e segurança de tipos. A arquitetura modular do NestJS facilita a adição de novas funcionalidades. React Native permite alcançar múltiplas plataformas com uma única base de código. Segundo Pressman and Maxim [2016], a seleção adequada de tecnologias é fundamental para o sucesso de projetos de software, devendo considerar fatores como maturidade, comunidade, documentação e alinhamento com objetivos do negócio. O InLight atende a esses critérios, utilizando tecnologias consolidadas e amplamente adotadas no mercado.

2.3 Sistemas de Informação e Integração

2.3.1 Conceitos Fundamentais de Sistemas de Informação

Sistemas de Informação (SI) são conjuntos organizados de elementos que coletam, processam, armazenam e distribuem informações para apoiar a tomada de decisão, coordenação, controle, análise e visualização em uma organização Laudon and Laudon [2020]. Segundo O'Brien and Marakas [2013], um SI pode ser definido como uma combinação organizada de pessoas, hardware, software, redes de comunicação, recursos de dados e políticas e procedimentos que armazenam, recuperam, transformam e disseminam informações.

Os componentes básicos de um SI incluem Stair and Reynolds [2018]:

- **Hardware:** equipamentos físicos utilizados para entrada, processamento e saída de dados;
- **Software:** programas e aplicações que controlam e coordenam o hardware;
- **Dados:** fatos brutos que são processados para gerar informação útil;
- **Procedimentos:** políticas e regras que governam a operação do sistema;
- **Pessoas:** usuários, desenvolvedores e gestores que interagem com o sistema;
- **Redes:** infraestrutura de comunicação que conecta componentes.

Para Turban and Volonino [2010], os SI são essenciais para o funcionamento de organizações modernas, pois transformam dados brutos em informações significativas que suportam decisões estratégicas, táticas e operacionais. A qualidade dessas decisões está diretamente relacionada à qualidade e à disponibilidade das informações fornecidas pelos SI.

2.3.2 Sistemas Legados e suas Limitações

Sistemas legados são aplicações de software antigas que continuam sendo críticas para as operações de uma organização, apesar de terem sido desenvolvidos com tecnologias ultrapassadas Bennett [1995]. Segundo Brooke and Ramage [1998], sistemas legados apresentam características particulares:

- Foram desenvolvidos há décadas, utilizando linguagens e paradigmas de programação antigos;
- Possuem documentação escassa ou desatualizada;
- Dependem de hardware e software descontinuados;
- Contêm regras de negócio críticas embutidas no código;
- São difíceis de modificar devido à complexidade acumulada;
- Apresentam alto custo de manutenção.

Pressman and Maxim [2016] destaca que sistemas legados representam um dilema para as organizações: embora sejam essenciais para operações críticas, sua manutenção é cada vez mais cara e arriscada. Além disso, vale a pena ser ressaltado que a falta de integração entre sistemas legados e aplicações mais modernas cria dificuldade na comunicação da informação que decorrentemente prejudicam a eficiência organizacional. Abordando a área do setor elétrico, muitas empresas ainda operam com sistemas desenvolvidos nas décadas de 1980 e1990, quando a estrutura tecnológica e os métodos de integração eram limitados. Esses sistemas, frequentemente desenvolvidos em linguagens como COBOL, Clipper ou Visual Basic, não foram projetados para se comunicar com aplicações modernas, resultando em processos manuais de transferência de dados Rezende [2017].

2.3.3 Fragmentação de Sistemas e seus Impactos

A fragmentação de sistemas ocorre quando uma organização utiliza múltiplas aplicações isoladas, cada uma gerenciando uma parte específica dos processos de negócio, sem integração adequada entre elas Laudon and Laudon [2020]. Esse cenário, comum em empresas que cresceram através de fusões ou com a implementação de novas tecnologias sem a conexão adequada, acaba gerando diversos problemas.

Redundância e Inconsistência de Dados

Quando dados são armazenados em múltiplos sistemas sem sincronização, surgem problemas de redundância e inconsistência Elmasri and Navathe [2016]. Por exemplo, informações sobre um mesmo contrato podem existir em diferentes versões em sistemas distintos, gerando conflitos sobre qual é a versão correta. Segundo Davenport [2018], essa inconsistência compromete a confiabilidade das informações e dificulta a tomada de decisões baseada em dados.

Retrabalho e Perda de Produtividade

A ausência de integração obriga colaboradores a realizar tarefas manuais repetitivas, como entrada de dados em múltiplos sistemas, exportação e importação de planilhas e conciliação manual de informações Stair and Reynolds [2018]. Segundo estudo de McKinsey & Company [2018], profissionais gastam em média 19

Dificuldade na Tomada de Decisão

A fragmentação impede uma visão de um todo dos processos organizacionais. Gestores precisam coletar manualmente informações de diversos sistemas, consolidá-las em planilhas e então realizar análises, processo que além de demorado está sujeito a erros Power [2002]. Para Sharda et al. [2014], a integração de dados é fundamental para sistemas de apoio à decisão eficazes.

Aumento de Custos Operacionais

Manter múltiplos sistemas não integrados implica em custos elevados de infraestrutura, licenciamento, treinamento e suporte técnico Turban and Volonino [2010]. Também vale ser

mencionado que todo esse trabalho manual entre sistemas, acaba consumindo recursos humanos que poderiam ser alocados em atividades gerando mais valor e decorrentemente lucro para a empresa.

Riscos de Segurança e Conformidade

Sistemas fragmentados dificultam a implementação de políticas de segurança uniformes e o controle de acesso Whitman and Mattord [2018]. Diferentes sistemas podem ter diferentes níveis de proteção, criando vulnerabilidades. Além disso, auditorias e demonstrações de conformidade com regulamentações tornam-se mais complexas quando dados estão dispersos Siponen and Vance [2010].

2.3.4 Integração de Sistemas: Conceitos e Abordagens

A integração de sistemas refere-se ao processo de conectar diferentes aplicações de software para que funcionem como um sistema unificado, compartilhando dados e processos de negócio Hohpe and Woolf [2003]. Segundo Linthicum [2000], integração não significa necessariamente substituir sistemas existentes, mas sim criar pontes que permitam comunicação eficiente entre eles.

Níveis de Integração

Ruh et al. [2000] identifica diferentes níveis de integração de sistemas:

- **Integração de Dados:** compartilhamento de dados entre sistemas através de bancos de dados compartilhados ou mecanismos de sincronização;
- **Integração de Aplicações:** sistemas comunicam-se através de APIs (Application Programming Interfaces), trocando mensagens e invocando funcionalidades remotamente;
- **Integração de Processos:** orquestração de processos de negócio que atravessam múltiplos sistemas;
- **Integração de Interface:** unificação da experiência do usuário através de portais ou dashboards que agregam funcionalidades de diversos sistemas.

Padrões e Tecnologias de Integração

Diversos padrões e tecnologias foram desenvolvidos para facilitar a integração de sistemas Hohpe and Woolf [2003]:

- **APIs RESTful:** interfaces baseadas no protocolo HTTP que seguem princípios REST, amplamente utilizadas por sua simplicidade e escalabilidade Fielding [2000];
- **Web Services:** serviços baseados em padrões como SOAP e WSDL, comuns em ambientes corporativos Alonso et al. [2004];
- **Message Queues:** sistemas de filas de mensagens (como RabbitMQ, Apache Kafka) que permitem comunicação assíncrona e desacoplada entre aplicações Videla and Williams [2011];
- **Enterprise Service Bus (ESB):** arquitetura que atua como barramento central para integração de múltiplos sistemas Chappell [2004];
- **Microserviços:** arquitetura que decompõe aplicações em serviços pequenos e independentes que se comunicam via APIs Newman [2015].

2.3.5 Unificação de Sistemas: Conceito e Benefícios

Enquanto integração refere-se à conexão entre sistemas mantendo-os separados, unificação implica em consolidar funcionalidades dispersas em uma plataforma única ou em um conjunto coeso de sistemas fortemente integrados Laudon and Laudon [2020], como é o caso do aplicativo InLight. A unificação vai além da integração ao oferecer:

Interface Única e Consistente

Usuários interagem com uma única aplicação que agrega funcionalidades antes dispersas, eliminando necessidade de alternar entre sistemas e aprender múltiplas interfaces Nielsen [1993]. Segundo Norman [2013], a consistência de interface reduz carga cognitiva e aumenta produtividade.

Modelo de Dados Unificado

Em vez de sincronizar dados entre sistemas distintos, a unificação estabelece um modelo de dados único e consistente, eliminando redundâncias e garantindo integridade referencial Elmasri and Navathe [2016]. Para Inmon [2005], a unificação de dados é essencial para criar uma "fonte única da verdade"(single source of truth) na organização.

Processos de Negócio Otimizados

A unificação permite redesenhar processos de negócio de forma holística, eliminando etapas desnecessárias e automatizando fluxos de trabalho que antes dependiam de intervenção manual Davenport [1993]. Segundo Hammer [2001], a unificação tecnológica deve ser acompanhada de reengenharia de processos para maximizar benefícios.

Manutenção Simplificada

Manter uma única plataforma é mais eficiente do que gerenciar múltiplos sistemas, reduzindo custos de infraestrutura, licenciamento e equipe técnica Pressman and Maxim [2016]. Além disso, atualizações e melhorias são implementadas uma única vez, beneficiando todos os módulos simultaneamente.

Agilidade e Inovação

Sistemas unificados, especialmente quando construídos com arquiteturas modernas e modulares, facilitam a adição de novas funcionalidades e a adaptação a mudanças de negócio Ross et al. [2006]. Para Westerman et al. [2014], agilidade tecnológica é fator crítico de sucesso em ambientes de transformação digital.

2.3.6 Desafios na Unificação de Sistemas

Apesar dos benefícios evidentes, a unificação de sistemas apresenta desafios significativos Linthicum [2000]:

- **Complexidade técnica:** migração de dados legados, mapeamento de regras de negócio e garantia de compatibilidade;

- **Resistência organizacional:** usuários acostumados com sistemas antigos podem resistir a mudanças Kotter [1996];
- **Custos e riscos:** investimento inicial elevado e riscos de falhas durante a transição Laudon and Laudon [2020];
- **Tempo de implementação:** projetos de unificação podem levar anos em organizações complexas;
- **Dependência de fornecedores:** escolha da plataforma adequada é crítica e difícil de reverter.

Segundo Markus and Tanis [2000], o sucesso de projetos de unificação depende não apenas de escolhas tecnológicas adequadas, mas também de gestão eficaz de mudanças, envolvimento de stakeholders e alinhamento com estratégia organizacional.

2.4 Gestão de Processos e Workflows

2.4.1 Conceitos de Processos de Negócio

Um processo de negócio é um conjunto estruturado de atividades inter-relacionadas que produzem um resultado de valor para o cliente ou organização Davenport [1993]. Segundo Hammer [2001], processos atravessam fronteiras funcionais e representam a forma como o trabalho realmente acontece nas organizações, independentemente da estrutura hierárquica formal.

vom Brocke and Rosemann [2007] categoriza processos em três tipos:

- **Processos primários:** agregam valor diretamente ao cliente (ex: atendimento, entrega de serviços);
- **Processos de suporte:** apoiam processos primários (ex: TI, recursos humanos, finanças);
- **Processos de gestão:** coordenam e controlam outros processos (ex: planejamento estratégico, governança).

A gestão de processos de negócio é uma abordagem projetada para identificar, desenhar, executar, documentar, medir, monitorar e controlar processos organizacionais para alcançar resultados consistentes e alinhados com objetivos estratégicos vom Brocke and Rosemann [2013].

2.4.2 Modelagem e Análise de Processos

A modelagem de processos envolve criar representações visuais que documentam como atividades são realizadas, quem as executa, quais sistemas são utilizados e quais informações são necessárias Aguilar-Savén [2004]. Processos comuns incluem:

- **BPMN (Business Process Model and Notation):** padrão internacional para modelagem de processos White [2004];
- **Fluxogramas:** diagramas simples que mostram sequência de atividades;
- **EPC (Event-driven Process Chain):** método que enfatiza eventos que disparam atividades;
- **UML Activity Diagrams:** diagramas da linguagem UML adaptados para processos.

Segundo Sharp and McDermott [2009], a modelagem permite identificar ineficiências, gargalos, redundâncias e oportunidades de melhoria antes de implementar mudanças. Análise de processos envolve examinar modelos para avaliar:

- Tempo de ciclo (cycle time) de processos;
- Identificação de atividades que não agregam valor;
- Pontos de falha e retrabalho;
- Utilização de recursos;
- Conformidade com regulamentações.

2.4.3 Reengenharia de Processos

Reengenharia de Processos de Negócio (Business Process Reengineering - BPR) é o repensar fundamental e redesenho radical de processos visando alcançar melhorias significativas em medidas críticas de desempenho Hammer and Champy [1993]. Diferentemente de melhoria contínua incremental, BPR busca transformações revolucionárias, ou seja, de grande impacto. Davenport [1993] identifica princípios fundamentais da reengenharia:

- **Organizar em torno de resultados, não tarefas:** atribuir responsabilidade por processos completos a equipes;
- **Fazer quem usa a informação processá-la:** eliminar intermediários desnecessários;
- **Tratar recursos geograficamente dispersos como centralizados:** usar tecnologia para coordenar;
- **Integrar atividades paralelas:** coordenar durante execução, não apenas no final;
- **Colocar decisões onde o trabalho é executado:** empoderar trabalhadores da linha de frente;
- **Capturar informação uma vez, na fonte:** eliminar reentrada de dados.

No contexto da unificação de sistemas, a reengenharia é essencial. Simplesmente automatizar processos ineficientes em novos sistemas não gera benefícios significativos Hammer [2001]. É necessário redesenhar processos aproveitando capacidades das novas tecnologias e o conceito de reengenharia e processos faz com que isso possa acontecer, de forma coesa e impactante.

2.4.4 Automação de Processos

Automação de processos refere-se ao uso de tecnologia para executar atividades repetitivas com mínima intervenção humana Lacity and Willcocks [2016]. Segundo vom Brocke and Rosemann [2013], benefícios da automação incluem:

- **Redução de erros:** sistemas automatizados executam tarefas consistentemente;
- **Maior velocidade:** atividades são concluídas mais rapidamente;
- **Disponibilidade contínua:** processos automatizados operam 24/7;
- **Melhor rastreabilidade:** sistemas registram cada etapa para auditoria;
- **Liberação de recursos humanos:** colaboradores focam em atividades estratégicas.

van der Aalst and van Hee [2003] categoriza tipos de automação:

- **Automação completa:** processo executado inteiramente por sistemas sem intervenção humana;

- **Automação parcial:** algumas etapas automatizadas, outras requerem decisão humana;
- **Automação assistida:** sistemas fornecem suporte mas humanos controlam execução.

Tecnologias modernas que suportam automação incluem:

- **Workflow engines:** motores que orquestram sequência de atividades;
- **RPA (Robotic Process Automation):** software que imita ações humanas em interfaces de sistemas Lacity and Willcocks [2016];
- **Business Rules Engines:** sistemas que aplicam regras de negócio automaticamente;
- **Inteligência Artificial:** algoritmos que aprendem e tomam decisões complexas Russell and Norvig [2020].

2.4.5 Workflows e Gestão de Tarefas

Workflow é a automação de um processo de negócio, no todo ou em parte, durante a qual documentos, informações ou tarefas são passadas de um participante para outro de acordo com um conjunto definido de regras Georgakopoulos et al. [1995].

Sistemas de workflow fornecem funcionalidades como:

- Definição de sequência de atividades e suas dependências;
- Atribuição automática de tarefas a usuários ou grupos;
- Notificações e lembretes sobre prazos;
- Dashboards de acompanhamento de status;
- Histórico completo de execução para auditoria;
- Escalação automática de tarefas atrasadas.

Segundo van der Aalst and van Hee [2003], workflows bem projetados reduzem tempo de ciclo de processos, melhoram coordenação entre equipes e aumentam transparência operacional. No contexto do InLight, funcionalidades de gestão de metas e tarefas exemplificam implementação de workflows digitais.

2.4.6 Redução de Retrabalho

Retrabalho refere-se à necessidade de refazer atividades devido a erros, falta de informação ou falhas de comunicação Womack and Jones [2003]. Segundo McKinsey & Company [2018], retrabalho consome em média 10-30

Causas comuns de retrabalho em ambientes com sistemas fragmentados incluem:

- **Entrada duplicada de dados:** mesma informação digitada em múltiplos sistemas;
- **Erros de transcrição:** falhas ao copiar dados manualmente entre sistemas;
- **Informações desatualizadas:** falta de sincronização causa uso de dados obsoletos;
- **Falta de visibilidade:** desconhecimento do status atual gera atividades redundantes;
- **Comunicação inadequada:** falhas na transmissão de informações entre equipes.

A unificação de sistemas reduz retrabalho através de Davenport [2018]:

- **Fonte única de dados:** eliminação de entrada duplicada;
- **Validações automáticas:** prevenção de erros na captura de dados;
- **Visibilidade em tempo real:** acesso instantâneo a informações atualizadas;
- **Workflows estruturados:** clareza sobre responsabilidades e prazos;
- **Histórico completo:** rastreabilidade de todas as ações realizadas.

A medição de desempenho é fundamental para gestão eficaz de processos Neely et al. [2005]. Indicadores-chave de desempenho (Key Performance Indicators - KPIs) comuns incluem:

- **Tempo de ciclo:** duração total desde início até conclusão do processo;
- **Throughput:** quantidade de processos concluídos por unidade de tempo;
- **Taxa de erro:** percentual de processos com falhas ou não conformidades;
- **Custo por processo:** recursos consumidos para executar uma instância;

- **Tempo de retrabalho:** tempo gasto corrigindo erros;
- **Satisfação do cliente:** percepção dos usuários sobre qualidade do serviço.

Segundo Kaplan and Norton [1996], sistemas de medição equilibrados avaliam múltiplas dimensões (financeira, cliente, processos internos, aprendizado) fornecendo visão holística do desempenho organizacional.

2.5 Transformação Digital no Setor Elétrico

2.5.1 Contexto do Setor Elétrico Brasileiro

O setor elétrico brasileiro é um dos maiores e mais complexos do mundo, em grande parte por sua territorialidade, as principais formas de energias utilizadas são hidráulica, eólica e nuclear. ANEEL [2022]. Segundo Tolmasquim [2016], o setor passou por profundas transformações nas últimas décadas:

- **Década de 1990:** privatização e abertura do mercado;
- **Anos 2000:** criação de novos marcos regulatórios após crise de 2001;
- **Anos 2010:** expansão do mercado livre e fontes renováveis;
- **Anos 2020:** digitalização e modernização da rede elétrica.

O setor é altamente regulado, com normas estabelecidas pela ANEEL (Agência Nacional de Energia Elétrica), operação coordenada pelo ONS (Operador Nacional do Sistema Elétrico) e comercialização regida pela CCEE Câ [2023].

2.5.2 CCEE - Câmara de Comercialização de Energia Elétrica

A CCEE é uma associação civil sem fins lucrativos que atua sob regulação e fiscalização da ANEEL, responsável por viabilizar a comercialização de energia elétrica no Sistema Interligado Nacional Câ [2023]. Suas principais funções incluem:

- **Contabilização:** apuração de diferenças entre energia contratada e consumida;
- **Liquidação financeira:** cálculo e cobrança de débitos e créditos no mercado de curto prazo;

- **Registro de contratos:** validação e armazenamento de contratos bilaterais;
- **Medição:** coleta e validação de dados de medição de energia;
- **Leilões:** suporte operacional a leilões de energia;
- **Garantias financeiras:** gestão de garantias para mitigar risco de inadimplência.

2.5.3 ACR e ACL: Ambientes de Contratação

O modelo brasileiro divide o mercado de energia em dois ambientes distintos ANEEL [2022]:

ACR - Ambiente de Contratação Regulada

No ACR, distribuidoras adquirem energia por meio de leilões regulados pela ANEEL para atender consumidores cativos (residenciais, comerciais e industriais de pequeno porte). Características do ACR:

- Contratos de longo prazo (15 a 30 anos);
- Preços definidos em leilões públicos;
- Garantia de suprimento para distribuidoras;
- Tarifas reguladas pela ANEEL;
- Menor exposição a volatilidade de preços.

ACL - Ambiente de Contratação Livre

No ACL, consumidores livres e especiais negociam diretamente com geradores e comercializadores, estabelecendo volume, preço e prazo livremente Câ [2023]. Características do ACL:

- Flexibilidade na negociação de contratos;
- Possibilidade de escolha do fornecedor;
- Contratos customizados conforme necessidade;
- Exposição a risco de preço no mercado de curto prazo;

- Gestão mais complexa exigindo expertise especializado.

Segundo de Souza Brito et al. [2018], o ACL tem crescido significativamente, representando aproximadamente 30

2.5.4 Desafios de Gestão de Contratos

A gestão de contratos no setor elétrico apresenta complexidades específicas Tolmasquim [2016]:

- **Volume de dados:** contratos envolvem medições horárias de múltiplos pontos;
- **Sazonalidade:** variações de demanda e geração ao longo do ano;
- **Reconciliação complexa:** necessidade de conciliar dados de medição, contabilização e faturamento;
- **Penalidades:** descumprimento de obrigações pode resultar em multas significativas;
- **Lastro e garantia física:** vendedores devem comprovar capacidade de geração;
- **Múltiplos submercados:** diferenças regionais de preço exigem gestão granular.

Para de Castro et al. [2015], empresas do setor necessitam de sistemas sofisticados capazes de integrar informações da CCEE, sistemas internos de medição, contratos comerciais e sistemas financeiros.

2.5.5 Obrigações e Prazos Regulatórios

Agentes do setor elétrico devem cumprir rigorosos prazos regulatórios estabelecidos pela ANEEL e CCEE ANEEL [2022]:

- Envio mensal de dados de medição;
- Registro de contratos com antecedência mínima;
- Prestação de garantias financeiras;
- Pagamento de encargos setoriais;

- Apresentação de demonstrativos contábeis;
- Atendimento a auditorias regulatórias.

O não cumprimento desses prazos pode resultar em Câ [2023]:

- Multas regulatórias;
- Suspensão temporária de operações;
- Perda de garantias financeiras depositadas;
- Impacto na reputação corporativa;
- Dificuldades em contratações futuras.

Segundo de Souza Brito et al. [2018], a gestão eficaz de prazos é crítica e demanda sistemas que forneçam alertas automatizados e visibilidade sobre status de obrigações pendentes.

2.5.6 Transformação Digital no Setor

O setor elétrico brasileiro está passando por acelerada transformação digital, impulsionada por EPE [2020]:

- **Smart Grids:** redes inteligentes com comunicação bidirecional;
- **Medidores inteligentes:** dispositivos que transmitem dados em tempo real;
- **IoT (Internet das Coisas):** sensores distribuídos pela rede;
- **Big Data e Analytics:** análise de grandes volumes de dados operacionais;
- **Inteligência Artificial:** previsão de demanda e manutenção preditiva;
- **Blockchain:** registro descentralizado de transações energéticas;
- **Computação em nuvem:** infraestrutura escalável e flexível.

Segundo Schwab [2016], a Quarta Revolução Industrial está transformando o setor energético, exigindo que empresas adotem tecnologias digitais para permanecerem competitivas. Para Westerman et al. [2014], transformação digital não é apenas sobre tecnologia, mas sobre reimaginar processos e modelos de negócio.

2.5.7 Integração de Sistemas no Contexto Regulatório

A complexidade regulatória do setor elétrico demanda integração sofisticada entre sistemas internos e externos de Castro et al. [2015]. Empresas precisam integrar:

- **Sistemas da CCEE:** para contabilização, liquidação e registro de contratos;
- **Sistemas de medição:** coleta de dados de consumo e geração;
- **ERP corporativo:** gestão financeira, contábil e administrativa;
- **Sistemas de trading:** negociação e gestão de contratos;
- **Sistemas de CRM:** relacionamento com clientes;
- **Business Intelligence:** análise e reporting gerencial.

Segundo Tolmasquim [2016], a falta de integração adequada resulta em processos manuais intensivos, retrabalho significativo e risco elevado de não conformidade regulatória. A unificação de sistemas, portanto, não é apenas desejável mas necessária para operação eficiente e conforme no setor elétrico brasileiro.

2.5.8 Casos de Sucesso e Benchmarks

Diversas empresas do setor elétrico brasileiro e internacional têm implementado projetos de unificação e modernização de sistemas com resultados positivos EPE [2020]:

- **Redução de até 40%** no tempo de processamento de obrigações regulatórias;
- **Diminuição de 60%** em erros de entrada de dados;
- **Melhoria de 50%** na acurácia de previsões de demanda;
- **Redução de 30%** em custos operacionais de TI;
- **Aumento de 70%** na satisfação dos usuários internos.

Esses resultados reforçam a importância estratégica da unificação tecnológica para empresas do setor, conforme argumentam Laudon and Laudon [2020] e Davenport [2018].

Capítulo 3

Revisão da Literatura / Trabalhos Relacionados

Capítulo 4

Proposta

Este capítulo tem como objetivo apresentar os conceitos, tencologias e práticas utilizadas no projeto. Irá ser abordado temas que se relacionam com o aplicativo InLight, com suas respectivas tecnologias, pontos positivos e negativos em práticas na implementação de unificação de sistemas e como deve ser realizado.

4.1 InLight

O InLight é um aplicativo de unificação de sistemas, desenvolvido em TypeScript, que oferece diversas aplicações, visto que agrupa as principais informações dos principais sistemas utilizados não somente no BackOffice ACR, mas como na LIGHT como um todo. É uma aplicação especifica para o mercado regulado podendo ser ampliado para o mercado livre.

Criado e desenvolvido para tratar informações da CCEE, Thunders e informações que só eram armazenadas em planilhas.

Ele foi estruturado em 3 principais módulos:

- Notas - agrupando contratos de quantidade, disponibilidade e cotas e todos os seus blocos, além de um contador na tela inicial para o próximo bloco.
- Metas - de acordo com a nova demanda, se foi necessário criar uma tela para metas de uma forma bem dinâmica.
- Obrigações - pagamentos chamados de especiais pelo modo que são tratados em seu pagamento.

4.2 Tecnologias

Abordando o desenvolvimento do aplicativo InLight, foi construído com um base de tecnologias que garantem modernidade e uma maior facilidade em alterações futuras. A partir deste ponto, irá ser apresentado as principais tecnologias utilizadas, organizadas de acordo com as suas funcionalidade e propósitos dentro do aplicativo.

4.2.1 Arquitetura da Aplicação

O InLight foi construído com uma arquitetura cliente-servidor, realizando assim uma boa divisão da interface do usuário (front-end) e a estrutura lógica de negócio (back-end). De acordo com, Pressman and Maxim [2016], realizando essa estrutura irá promover uma maior facilidade na manutenção e na escalabilidade, além de facilitar que equipes diferentes trabalhem no projeto.

4.2.2 Front-end: React Native

React Native e TypeScript

Falando sobre o front-end do InLight, ele foi desenvolvido em React Native, que é um Framework JavaScript mantido pelo Facebook (Meta), esta tecnologia permite o desenvolvimento de aplicações mobile para Android e iOS a partir de uma única base de código Eisenman [2015]. Partindo deste viés, a escolha do React Native se justifica por:

- **Desenvolvimento multiplataforma:** como não é necessário manter duas bases separadamente, reduz consideravelmente o tempo e custo de desenvolvimento
- **Hot Reload:** permite visualizar atualizações no código em tempo real, assim tornando o processo de desenvolvimento mais rápido.
- **Comunidade ativa:** Comunidade bastante engajada e boa diversidade de bibliotecas.

Ademais, o projeto foi desenvolvido em TypeScript, que acrescenta a tipagem estática opcional, já que o JavaScript é uma linguagem de tipagem dinâmica. Seguindo essa filosofia, proporciona uma maior segurança no desenvolvimento e facilitará a manutenção no código como mencionado por Bierman et al. [2014]. Seguindo a análise de, Microsoft [2020], a utilização da tipagem estática reduz a chance de ocorrer erros durante a execução e melhora a experiência de desenvolvimento através autocompletar e refatoração.

Gerenciamento de Estado e Navegação

Para o gerenciamento de estado global, o InLight utiliza a Context API do React, este recurso permite compartilhar dados entre componentes sem necessidade de passar propriedades

manualmente através das árvores de componentes Facebook [2023a]. O contexto de autenticação (AuthContext) é responsável no código por gerenciar informações do usuário logado, tokens de acesso e estado de autenticação. Abordando a troca entre telas, foi utilizado a biblioteca React Navigation, falando mais especificamente foi utilizado o padrão Stack Navigator. Essa biblioteca oferece navegação nativa, transições suaves e gerenciamento de histórico de navegação React Navigation [2023]. Na parte de locação de dados, como por exemplo tokens de autenticação, foi utilizado AsyncStorage, que fornece um sistema de armazenamento chave-valor assíncrono e não criptografado Facebook [2023b].

Componentes e Interface de Usuário

A interface do usuário foi desenvolvida utilizando os componentes básicos do React Native (View, Text, TouchableOpacity, FlatList, ScrollView), complementados por bibliotecas mais específicas:

- **@react-native-community/slider:** componente de controle deslizante para ajuste de progresso de metas, sendo bem dinâmica em todas as telas;
- **@react-native-picker/picker:** seletor de opções para escolha de papéis de usuário;
- **expo-document-picker:** seleção de documentos do dispositivo para funcionalidade de upload, utilizado na parte de pendências.

4.2.3 Back-end: NestJS e Node.js

Framework NestJS

O back-end do InLight é desenvolvido em NestJS, um framework progressivo para construção de aplicações server-side eficientes e escaláveis NestJS [2023]. O NestJS é construído em Node.js e utiliza Typescript como linguagem padrão, garantindo uma boa conexão com o front-end. As principais características do NestJS que justificam sua adoção incluem:

- **Arquitetura modular:** organização do código em módulos independentes e reutilizáveis;
- **Injeção de dependências:** padrão que facilita testes e manutenção do código;
- **Suporte a TypeScript:** aproveitamento o máximo dos benefícios da tipagem estática;

- **Decorators:** metaprogramação que torna o código mais declarativo e legível;
- **Estrutura baseada em Angular:** familiaridade para desenvolvedores que conhecem frameworks modernos.

Arquitetura em Camadas

O back-end segue uma arquitetura em três camadas bem definidas, conforme recomendado por Fowler [2002]:

1. **Camada de Apresentação (Controllers):** responsável por receber requisições HTTP, validar entradas e retornar respostas adequadas. Utiliza em seu código decorators como por exemplo: @Controller, @Get, @Post, @Patch e @Delete para definir rotas e métodos HTTP;
2. **Camada de Negócio (Services):** contém a lógica de negócio do aplicativo, implementando regras, validações e orquestrando operações entre diferentes entidades;
3. **Camada de Dados (Repositories):** responsável pela comunicação com o banco de dados, abstraindo detalhes de persistência através do padrão Repository.

Esta separação promove o princípio da responsabilidade única (Single Responsibility Principle) do SOLID Martin [2008], facilitando manutenção, testes e evolução do sistema.

4.2.4 Banco de Dados e Persistência

PostgreSQL

O InLight utiliza PostgreSQL como sistema gerenciador de banco de dados. Este banco dados é um open-source robusto, conhecido por sua confiabilidade, integridade de dados e conformidade com padrões SQL PostgreSQL Global Development Group [2023]. Segundo Obe and Hsu [2017], PostgreSQL oferece recursos avançados como:

- Suporte completo a transações ACID (Atomicidade, Consistência, Isolamento, Durabilidade);
- Tipos de dados complexos e extensíveis;
- Índices avançados para otimização de consultas;

- Suporte a JSON e dados semi-estruturados;
- Replicação e alta disponibilidade.

TypeORM

Para a comunicação entre a aplicação e o banco de dados, utiliza-se TypeORM, um Object-Relational Mapping (ORM) que permite trabalhar com entidades de banco de dados como objetos TypeScript TypeORM [2023]. O TypeORM oferece:

- **Query Builder:** construção de consultas complexas de forma programática e type-safe;
- **Migrations:** versionamento e controle de mudanças no schema do banco;
- **Relacionamentos:** definição declarativa de relações entre entidades (OneToMany, ManyToOne, ManyToMany);
- **Lazy Loading e Eager Loading:** estratégias de carregamento de relacionamentos.

O uso de ORM reduz significativamente a quantidade de código SQL escrito manualmente, previne vulnerabilidades como SQL Injection e facilita a portabilidade entre diferentes SGBDs Ambler [2003].

4.2.5 Autenticação e Segurança

JSON Web Tokens (JWT)

O sistema de autenticação do InLight baseia-se em JSON Web Tokens (JWT), um padrão aberto (RFC 7519) para transmissão segura de informações entre partes como um objeto JSON Jones et al. [2015]. O fluxo de autenticação funciona da seguinte forma:

1. Usuário envia credenciais (email e senha) para o endpoint de login;
2. Servidor valida as credenciais e, se válidas, gera um token JWT contendo informações do usuário;
3. Cliente armazena o token localmente e o envia no header Authorization de requisições subsequentes;
4. Servidor valida o token em cada requisição protegida.

Os benefícios do JWT incluem:

- **Stateless:** servidor não precisa armazenar sessões, facilitando escalabilidade;
- **Autocontido:** token contém todas as informações necessárias para autenticação;
- **Compacto:** pode ser transmitido via URL, POST ou HTTP header;
- **Seguro:** pode ser assinado digitalmente garantindo integridade.

Passport.js e Estratégias de Autenticação

A implementação da autenticação utiliza Passport.js, um middleware de autenticação flexível e modular para Node.js Jared Hanson [2023]. É aplicado passport-jwt para validação de tokens JWT. O NestJS integra o Passport através do modulo @nestjs/passport, fornecendo Guards que protegem rotas específicas.

Criptografia de Senhas

Para o armazenamento seguro de senhas, utiliza-se a biblioteca bcrypt, que implementa o algoritmo de hashing bcrypt baseado no Blowfish cipher Provos and Mazières [1999]. Características importantes:

- **Hash com salt:** cada senha recebe um salt único, prevenindo ataques de rainbow table;
- **Função de custo adaptável:** permite aumentar a complexidade computacional ao longo do tempo;
- **Resistente a ataques de força bruta:** custo computacional alto, porém dificulta quebrar senhas.

Controle de Acesso Baseado em Papéis (RBAC)

O InLight implementa RBAC (Role-Based Access Control) com três níveis de acesso:

- **Admin:** acesso total ao sistema, incluindo cadastro de usuários e importação de dados;
- **Manager:** acesso intermediário com permissões de gestão;
- **User:** acesso básico às funcionalidades essenciais.

Segundo Ferraiolo et al. [2001], RBAC simplifica a administração de permissões e garante conformidade com políticas de segurança organizacionais.

4.2.6 Validação e Tratamento de Dados

Data Transfer Objects (DTOs)

No aplicativo InLight o padrão DTO é utilizado para transferência de dados entre camadas Fowler [2002]. DTOs são objetos simples que transportam dados, sem conter lógica de negócio, que é principalmente no service. No contexto do projeto, existem DTOs específicos para:

- **CreateDTO:** dados necessários para criação de entidades;
- **UpdateDTO:** dados para atualização (geralmente PartialType do CreateDTO);
- **FilterDTO:** parâmetros de filtro para consultas;
- **ResponseDTO:** formatação de respostas da API.

Class-validator

A validação de dados de entrada é realizada através da biblioteca class-validator, que utiliza decorators TypeScript para definir regras de validação. Exemplos de decorators utilizados:

- @IsNotEmpty: campo obrigatório;
- @IsEmail: validação de formato de email;
- @IsDateString: validação de formato de data;
- @IsNumber: validação de tipo numérico;
- @IsString: validação de tipo string.

ValidationPipe

O NestJS fornece o recurso ValidationPipe, um pipe global que automaticamente valida payloads de requisições nos DTOs definidos. Configurado no arquivo main.ts com as opções:

- **whitelist: true** - remove propriedades não definidas no DTO;
- **forbidNonWhitelisted: true** - rejeita requisições com propriedades extras;
- **transform: true** - converte automaticamente tipos primitivos.

Essa abordagem garante que apenas dados válidos cheguem às camadas de negócio e dados.

4.2.7 Interceptors e Exception Filters

Response Interceptor

O ResponseInterceptor padroniza o formato de todas as respostas da API, encapsulando dados em uma estrutura consistente. Isso facilita o tratamento no front-end e melhora a experiência do desenvolvedor NestJS [2023].

HttpExceptionFilter

O HttpExceptionFilter customizado captura exceções HTTP e formata respostas de erro de maneira uniforme, incluindo:

- Código de status HTTP apropriado;
- Mensagem de erro descritiva;
- Timestamp da ocorrência;
- Path da requisição;
- Detalhes adicionais quando apropriado.

Segundo Pressman and Maxim [2016], tratamento adequado de exceções é fundamental para robustez e manutenibilidade de sistemas.

4.2.8 Processamento de Arquivos Excel

Uma funcionalidade muito importante do InLight é a importação de através de arquivos Excel (.xlsx), especialmente para obrigações futuras. O processo inclui:

- **Upload via FormData:** suporte a multipart/form-data;
- **Conversão de datas seriais:** Excel armazena datas como números seriais desde 1900-01-01, exigindo conversão para formato padrão;
- **Validação de dados:** verificação de formato e integridade antes de persistir;
- **Feedback ao usuário:** notificação de sucesso ou erro detalhado.

4.2.9 Comunicação HTTP e CORS

RESTful API

A API do InLight segue os princípios REST (Representational State Transfer) definidos por Fielding [2000]:

- **Cliente-Servidor:** separação clara de responsabilidades;
- **Stateless:** cada requisição contém todas as informações necessárias;
- **Cacheable:** respostas podem ser cacheadas quando apropriado;
- **Interface uniforme:** uso consistente de métodos HTTP;
- **Sistema em camadas:** arquitetura pode incluir intermediários.

CORS (Cross-Origin Resource Sharing)

O back-end habilita CORS para permitir que o aplicativo mobile acesse a API. A configuração no main.ts especifica:

- Origins permitidas;
- Métodos HTTP aceitos (GET, POST, PATCH, DELETE);
- Suporte a credenciais;

Essa configuração é essencial para comunicação segura entre diferentes origens W3C [2014].

4.2.10 Testes Automatizados

O aplicativo utiliza Jest como framework de testes, permitindo:

- **Testes unitários:** validação de componentes isolados;
- **Testes de integração:** verificação de interação entre módulos;
- **Mocking:** simulação de dependências externas;
- **Coverage reports:** análise de cobertura de código.

O NestJS fornece @nestjs/testing, que facilita a criação de módulos de teste com injeção de dependências NestJS [2023].

4.2.11 Considerações sobre a Stack Tecnológica

A escolha das tecnologias empregadas no InLight reflete uma busca por modernidade, escalabilidade e manutenibilidade. O uso de TypeScript em toda a sua estrutura garante consistência e segurança de tipos. A arquitetura modular do NestJS facilita a adição de novas funcionalidades. React Native permite alcançar múltiplas plataformas com uma única base de código. Segundo Pressman and Maxim [2016], a seleção adequada de tecnologias é fundamental para o sucesso de projetos de software, devendo considerar fatores como maturidade, comunidade, documentação e alinhamento com objetivos do negócio. O InLight atende a esses critérios, utilizando tecnologias consolidadas e amplamente adotadas no mercado.

4.3 Sistemas de Informação e Integração

4.3.1 Conceitos Fundamentais de Sistemas de Informação

Sistemas de Informação (SI) são conjuntos organizados de elementos que coletam, processam, armazenam e distribuem informações para apoiar a tomada de decisão, coordenação, controle, análise e visualização em uma organização Laudon and Laudon [2020]. Segundo O'Brien and Marakas [2013], um SI pode ser definido como uma combinação organizada de pessoas, hardware, software, redes de comunicação, recursos de dados e políticas e procedimentos que armazenam, recuperam, transformam e disseminam informações.

Os componentes básicos de um SI incluem Stair and Reynolds [2018]:

- **Hardware:** equipamentos físicos utilizados para entrada, processamento e saída de dados;
- **Software:** programas e aplicações que controlam e coordenam o hardware;
- **Dados:** fatos brutos que são processados para gerar informação útil;
- **Procedimentos:** políticas e regras que governam a operação do sistema;
- **Pessoas:** usuários, desenvolvedores e gestores que interagem com o sistema;
- **Redes:** infraestrutura de comunicação que conecta componentes.

Para Turban and Volonino [2010], os SI são essenciais para o funcionamento de organizações modernas, pois transformam dados brutos em informações significativas que suportam decisões estratégicas, táticas e operacionais. A qualidade dessas decisões está diretamente relacionada à qualidade e à disponibilidade das informações fornecidas pelos SI.

4.3.2 Sistemas Legados e suas Limitações

Sistemas legados são aplicações de software antigas que continuam sendo críticas para as operações de uma organização, apesar de terem sido desenvolvidos com tecnologias ultrapassadas Bennett [1995]. Segundo Brooke and Ramage [1998], sistemas legados apresentam características particulares:

- Foram desenvolvidos há décadas, utilizando linguagens e paradigmas de programação antigos;
- Possuem documentação escassa ou desatualizada;
- Dependem de hardware e software descontinuados;
- Contêm regras de negócio críticas embutidas no código;
- São difíceis de modificar devido à complexidade acumulada;
- Apresentam alto custo de manutenção.

Pressman and Maxim [2016] destaca que sistemas legados representam um dilema para as organizações: embora sejam essenciais para operações críticas, sua manutenção é cada vez mais cara e arriscada. Além disso, vale a pena ser ressaltado que a falta de integração entre sistemas legados e aplicações mais modernas cria dificuldade na comunicação da informação que decorrentemente prejudicam a eficiência organizacional. Abordando a área do setor elétrico, muitas empresas ainda operam com sistemas desenvolvidos nas décadas de 1980 e1990, quando a estrutura tecnológica e os métodos de integração eram limitados. Esses sistemas, frequentemente desenvolvidos em linguagens como COBOL, Clipper ou Visual Basic, não foram projetados para se comunicar com aplicações modernas, resultando em processos manuais de transferência de dados Rezende [2017].

4.3.3 Fragmentação de Sistemas e seus Impactos

A fragmentação de sistemas ocorre quando uma organização utiliza múltiplas aplicações isoladas, cada uma gerenciando uma parte específica dos processos de negócio, sem integração adequada entre elas Laudon and Laudon [2020]. Esse cenário, comum em empresas que cresceram através de fusões ou com a implementação de novas tecnologias sem a conexão adequada, acaba gerando diversos problemas.

Redundância e Inconsistência de Dados

Quando dados são armazenados em múltiplos sistemas sem sincronização, surgem problemas de redundância e inconsistência Elmasri and Navathe [2016]. Por exemplo, informações sobre um mesmo contrato podem existir em diferentes versões em sistemas distintos, gerando conflitos sobre qual é a versão correta. Segundo Davenport [2018], essa inconsistência compromete a confiabilidade das informações e dificulta a tomada de decisões baseada em dados.

Retrabalho e Perda de Produtividade

A ausência de integração obriga colaboradores a realizar tarefas manuais repetitivas, como entrada de dados em múltiplos sistemas, exportação e importação de planilhas e conciliação manual de informações Stair and Reynolds [2018]. Segundo estudo de McKinsey & Company [2018], profissionais gastam em média 19

Dificuldade na Tomada de Decisão

A fragmentação impede uma visão de um todo dos processos organizacionais. Gestores precisam coletar manualmente informações de diversos sistemas, consolidá-las em planilhas e então realizar análises, processo que além de demorado está sujeito a erros Power [2002]. Para Sharda et al. [2014], a integração de dados é fundamental para sistemas de apoio à decisão eficazes.

Aumento de Custos Operacionais

Manter múltiplos sistemas não integrados implica em custos elevados de infraestrutura, licenciamento, treinamento e suporte técnico Turban and Volonino [2010]. Também vale ser

mencionado que todo esse trabalho manual entre sistemas, acaba consumindo recursos humanos que poderiam ser alocados em atividades gerando mais valor e decorrentemente lucro para a empresa.

Riscos de Segurança e Conformidade

Sistemas fragmentados dificultam a implementação de políticas de segurança uniformes e o controle de acesso Whitman and Mattord [2018]. Diferentes sistemas podem ter diferentes níveis de proteção, criando vulnerabilidades. Além disso, auditorias e demonstrações de conformidade com regulamentações tornam-se mais complexas quando dados estão dispersos Siponen and Vance [2010].

4.3.4 Integração de Sistemas: Conceitos e Abordagens

A integração de sistemas refere-se ao processo de conectar diferentes aplicações de software para que funcionem como um sistema unificado, compartilhando dados e processos de negócio Hohpe and Woolf [2003]. Segundo Linthicum [2000], integração não significa necessariamente substituir sistemas existentes, mas sim criar pontes que permitam comunicação eficiente entre eles.

Níveis de Integração

Ruh et al. [2000] identifica diferentes níveis de integração de sistemas:

- **Integração de Dados:** compartilhamento de dados entre sistemas através de bancos de dados compartilhados ou mecanismos de sincronização;
- **Integração de Aplicações:** sistemas comunicam-se através de APIs (Application Programming Interfaces), trocando mensagens e invocando funcionalidades remotamente;
- **Integração de Processos:** orquestração de processos de negócio que atravessam múltiplos sistemas;
- **Integração de Interface:** unificação da experiência do usuário através de portais ou dashboards que agregam funcionalidades de diversos sistemas.

Padrões e Tecnologias de Integração

Diversos padrões e tecnologias foram desenvolvidos para facilitar a integração de sistemas Hohpe and Woolf [2003]:

- **APIs RESTful:** interfaces baseadas no protocolo HTTP que seguem princípios REST, amplamente utilizadas por sua simplicidade e escalabilidade Fielding [2000];
- **Web Services:** serviços baseados em padrões como SOAP e WSDL, comuns em ambientes corporativos Alonso et al. [2004];
- **Message Queues:** sistemas de filas de mensagens (como RabbitMQ, Apache Kafka) que permitem comunicação assíncrona e desacoplada entre aplicações Videla and Williams [2011];
- **Enterprise Service Bus (ESB):** arquitetura que atua como barramento central para integração de múltiplos sistemas Chappell [2004];
- **Microserviços:** arquitetura que decompõe aplicações em serviços pequenos e independentes que se comunicam via APIs Newman [2015].

4.3.5 Unificação de Sistemas: Conceito e Benefícios

Enquanto integração refere-se à conexão entre sistemas mantendo-os separados, unificação implica em consolidar funcionalidades dispersas em uma plataforma única ou em um conjunto coeso de sistemas fortemente integrados Laudon and Laudon [2020], como é o caso do aplicativo InLight. A unificação vai além da integração ao oferecer:

Interface Única e Consistente

Usuários interagem com uma única aplicação que agrega funcionalidades antes dispersas, eliminando necessidade de alternar entre sistemas e aprender múltiplas interfaces Nielsen [1993]. Segundo Norman [2013], a consistência de interface reduz carga cognitiva e aumenta produtividade.

Modelo de Dados Unificado

Em vez de sincronizar dados entre sistemas distintos, a unificação estabelece um modelo de dados único e consistente, eliminando redundâncias e garantindo integridade referencial Elmasri and Navathe [2016]. Para Inmon [2005], a unificação de dados é essencial para criar uma "fonte única da verdade"(single source of truth) na organização.

Processos de Negócio Otimizados

A unificação permite redesenhar processos de negócio de forma holística, eliminando etapas desnecessárias e automatizando fluxos de trabalho que antes dependiam de intervenção manual Davenport [1993]. Segundo Hammer [2001], a unificação tecnológica deve ser acompanhada de reengenharia de processos para maximizar benefícios.

Manutenção Simplificada

Manter uma única plataforma é mais eficiente do que gerenciar múltiplos sistemas, reduzindo custos de infraestrutura, licenciamento e equipe técnica Pressman and Maxim [2016]. Além disso, atualizações e melhorias são implementadas uma única vez, beneficiando todos os módulos simultaneamente.

Agilidade e Inovação

Sistemas unificados, especialmente quando construídos com arquiteturas modernas e modulares, facilitam a adição de novas funcionalidades e a adaptação a mudanças de negócio Ross et al. [2006]. Para Westerman et al. [2014], agilidade tecnológica é fator crítico de sucesso em ambientes de transformação digital.

4.3.6 Desafios na Unificação de Sistemas

Apesar dos benefícios evidentes, a unificação de sistemas apresenta desafios significativos Linthicum [2000]:

- **Complexidade técnica:** migração de dados legados, mapeamento de regras de negócio e garantia de compatibilidade;

- **Resistência organizacional:** usuários acostumados com sistemas antigos podem resistir a mudanças Kotter [1996];
- **Custos e riscos:** investimento inicial elevado e riscos de falhas durante a transição Laudon and Laudon [2020];
- **Tempo de implementação:** projetos de unificação podem levar anos em organizações complexas;
- **Dependência de fornecedores:** escolha da plataforma adequada é crítica e difícil de reverter.

Segundo Markus and Tanis [2000], o sucesso de projetos de unificação depende não apenas de escolhas tecnológicas adequadas, mas também de gestão eficaz de mudanças, envolvimento de stakeholders e alinhamento com estratégia organizacional.

4.4 Gestão de Processos e Workflows

4.4.1 Conceitos de Processos de Negócio

Um processo de negócio é um conjunto estruturado de atividades inter-relacionadas que produzem um resultado de valor para o cliente ou organização Davenport [1993]. Segundo Hammer [2001], processos atravessam fronteiras funcionais e representam a forma como o trabalho realmente acontece nas organizações, independentemente da estrutura hierárquica formal.

vom Brocke and Rosemann [2007] categoriza processos em três tipos:

- **Processos primários:** agregam valor diretamente ao cliente (ex: atendimento, entrega de serviços);
- **Processos de suporte:** apoiam processos primários (ex: TI, recursos humanos, finanças);
- **Processos de gestão:** coordenam e controlam outros processos (ex: planejamento estratégico, governança).

A gestão de processos de negócio é uma abordagem projetada para identificar, desenhar, executar, documentar, medir, monitorar e controlar processos organizacionais para alcançar resultados consistentes e alinhados com objetivos estratégicos vom Brocke and Rosemann [2013].

4.4.2 Modelagem e Análise de Processos

A modelagem de processos envolve criar representações visuais que documentam como atividades são realizadas, quem as executa, quais sistemas são utilizados e quais informações são necessárias Aguilar-Savén [2004]. Processos comuns incluem:

- **BPMN (Business Process Model and Notation):** padrão internacional para modelagem de processos White [2004];
- **Fluxogramas:** diagramas simples que mostram sequência de atividades;
- **EPC (Event-driven Process Chain):** método que enfatiza eventos que disparam atividades;
- **UML Activity Diagrams:** diagramas da linguagem UML adaptados para processos.

Segundo Sharp and McDermott [2009], a modelagem permite identificar ineficiências, gargalos, redundâncias e oportunidades de melhoria antes de implementar mudanças. Análise de processos envolve examinar modelos para avaliar:

- Tempo de ciclo (cycle time) de processos;
- Identificação de atividades que não agregam valor;
- Pontos de falha e retrabalho;
- Utilização de recursos;
- Conformidade com regulamentações.

4.4.3 Reengenharia de Processos

Reengenharia de Processos de Negócio (Business Process Reengineering - BPR) é o repensar fundamental e redesenho radical de processos visando alcançar melhorias significativas em medidas críticas de desempenho Hammer and Champy [1993]. Diferentemente de melhoria contínua incremental, BPR busca transformações revolucionárias, ou seja, de grande impacto. Davenport [1993] identifica princípios fundamentais da reengenharia:

- **Organizar em torno de resultados, não tarefas:** atribuir responsabilidade por processos completos a equipes;
- **Fazer quem usa a informação processá-la:** eliminar intermediários desnecessários;
- **Tratar recursos geograficamente dispersos como centralizados:** usar tecnologia para coordenar;
- **Integrar atividades paralelas:** coordenar durante execução, não apenas no final;
- **Colocar decisões onde o trabalho é executado:** empoderar trabalhadores da linha de frente;
- **Capturar informação uma vez, na fonte:** eliminar reentrada de dados.

No contexto da unificação de sistemas, a reengenharia é essencial. Simplesmente automatizar processos ineficientes em novos sistemas não gera benefícios significativos Hammer [2001]. É necessário redesenhar processos aproveitando capacidades das novas tecnologias e o conceito de reengenharia e processos faz com que isso possa acontecer, de forma coesa e impactante.

4.4.4 Automação de Processos

Automação de processos refere-se ao uso de tecnologia para executar atividades repetitivas com mínima intervenção humana Lacity and Willcocks [2016]. Segundo vom Brocke and Rosemann [2013], benefícios da automação incluem:

- **Redução de erros:** sistemas automatizados executam tarefas consistentemente;
- **Maior velocidade:** atividades são concluídas mais rapidamente;
- **Disponibilidade contínua:** processos automatizados operam 24/7;
- **Melhor rastreabilidade:** sistemas registram cada etapa para auditoria;
- **Liberação de recursos humanos:** colaboradores focam em atividades estratégicas.

van der Aalst and van Hee [2003] categoriza tipos de automação:

- **Automação completa:** processo executado inteiramente por sistemas sem intervenção humana;

- **Automação parcial:** algumas etapas automatizadas, outras requerem decisão humana;
- **Automação assistida:** sistemas fornecem suporte mas humanos controlam execução.

Tecnologias modernas que suportam automação incluem:

- **Workflow engines:** motores que orquestram sequência de atividades;
- **RPA (Robotic Process Automation):** software que imita ações humanas em interfaces de sistemas Lacity and Willcocks [2016];
- **Business Rules Engines:** sistemas que aplicam regras de negócio automaticamente;
- **Inteligência Artificial:** algoritmos que aprendem e tomam decisões complexas Russell and Norvig [2020].

4.4.5 Workflows e Gestão de Tarefas

Workflow é a automação de um processo de negócio, no todo ou em parte, durante a qual documentos, informações ou tarefas são passadas de um participante para outro de acordo com um conjunto definido de regras Georgakopoulos et al. [1995].

Sistemas de workflow fornecem funcionalidades como:

- Definição de sequência de atividades e suas dependências;
- Atribuição automática de tarefas a usuários ou grupos;
- Notificações e lembretes sobre prazos;
- Dashboards de acompanhamento de status;
- Histórico completo de execução para auditoria;
- Escalação automática de tarefas atrasadas.

Segundo van der Aalst and van Hee [2003], workflows bem projetados reduzem tempo de ciclo de processos, melhoram coordenação entre equipes e aumentam transparência operacional. No contexto do InLight, funcionalidades de gestão de metas e tarefas exemplificam implementação de workflows digitais.

4.4.6 Redução de Retrabalho

Retrabalho refere-se à necessidade de refazer atividades devido a erros, falta de informação ou falhas de comunicação Womack and Jones [2003]. Segundo McKinsey & Company [2018], retrabalho consome em média 10-30

Causas comuns de retrabalho em ambientes com sistemas fragmentados incluem:

- **Entrada duplicada de dados:** mesma informação digitada em múltiplos sistemas;
- **Erros de transcrição:** falhas ao copiar dados manualmente entre sistemas;
- **Informações desatualizadas:** falta de sincronização causa uso de dados obsoletos;
- **Falta de visibilidade:** desconhecimento do status atual gera atividades redundantes;
- **Comunicação inadequada:** falhas na transmissão de informações entre equipes.

A unificação de sistemas reduz retrabalho através de Davenport [2018]:

- **Fonte única de dados:** eliminação de entrada duplicada;
- **Validações automáticas:** prevenção de erros na captura de dados;
- **Visibilidade em tempo real:** acesso instantâneo a informações atualizadas;
- **Workflows estruturados:** clareza sobre responsabilidades e prazos;
- **Histórico completo:** rastreabilidade de todas as ações realizadas.

A medição de desempenho é fundamental para gestão eficaz de processos Neely et al. [2005]. Indicadores-chave de desempenho (Key Performance Indicators - KPIs) comuns incluem:

- **Tempo de ciclo:** duração total desde início até conclusão do processo;
- **Throughput:** quantidade de processos concluídos por unidade de tempo;
- **Taxa de erro:** percentual de processos com falhas ou não conformidades;
- **Custo por processo:** recursos consumidos para executar uma instância;

- **Tempo de retrabalho:** tempo gasto corrigindo erros;
- **Satisfação do cliente:** percepção dos usuários sobre qualidade do serviço.

Segundo Kaplan and Norton [1996], sistemas de medição equilibrados avaliam múltiplas dimensões (financeira, cliente, processos internos, aprendizado) fornecendo visão holística do desempenho organizacional.

4.5 Transformação Digital no Setor Elétrico

4.5.1 Contexto do Setor Elétrico Brasileiro

O setor elétrico brasileiro é um dos maiores e mais complexos do mundo, em grande parte por sua territorialidade, as principais formas de energias utilizadas são hidráulica, eólica e nuclear. ANEEL [2022]. Segundo Tolmasquim [2016], o setor passou por profundas transformações nas últimas décadas:

- **Década de 1990:** privatização e abertura do mercado;
- **Anos 2000:** criação de novos marcos regulatórios após crise de 2001;
- **Anos 2010:** expansão do mercado livre e fontes renováveis;
- **Anos 2020:** digitalização e modernização da rede elétrica.

O setor é altamente regulado, com normas estabelecidas pela ANEEL (Agência Nacional de Energia Elétrica), operação coordenada pelo ONS (Operador Nacional do Sistema Elétrico) e comercialização regida pela CCEE Câ [2023].

4.5.2 CCEE - Câmara de Comercialização de Energia Elétrica

A CCEE é uma associação civil sem fins lucrativos que atua sob regulação e fiscalização da ANEEL, responsável por viabilizar a comercialização de energia elétrica no Sistema Interligado Nacional Câ [2023]. Suas principais funções incluem:

- **Contabilização:** apuração de diferenças entre energia contratada e consumida;
- **Liquidação financeira:** cálculo e cobrança de débitos e créditos no mercado de curto prazo;

- **Registro de contratos:** validação e armazenamento de contratos bilaterais;
- **Medição:** coleta e validação de dados de medição de energia;
- **Leilões:** suporte operacional a leilões de energia;
- **Garantias financeiras:** gestão de garantias para mitigar risco de inadimplência.

4.5.3 ACR e ACL: Ambientes de Contratação

O modelo brasileiro divide o mercado de energia em dois ambientes distintos ANEEL [2022]:

ACR - Ambiente de Contratação Regulada

No ACR, distribuidoras adquirem energia por meio de leilões regulados pela ANEEL para atender consumidores cativos (residenciais, comerciais e industriais de pequeno porte). Características do ACR:

- Contratos de longo prazo (15 a 30 anos);
- Preços definidos em leilões públicos;
- Garantia de suprimento para distribuidoras;
- Tarifas reguladas pela ANEEL;
- Menor exposição a volatilidade de preços.

ACL - Ambiente de Contratação Livre

No ACL, consumidores livres e especiais negociam diretamente com geradores e comercializadores, estabelecendo volume, preço e prazo livremente Câ [2023]. Características do ACL:

- Flexibilidade na negociação de contratos;
- Possibilidade de escolha do fornecedor;
- Contratos customizados conforme necessidade;
- Exposição a risco de preço no mercado de curto prazo;

- Gestão mais complexa exigindo expertise especializado.

Segundo de Souza Brito et al. [2018], o ACL tem crescido significativamente, representando aproximadamente 30

4.5.4 Desafios de Gestão de Contratos

A gestão de contratos no setor elétrico apresenta complexidades específicas Tolmasquim [2016]:

- **Volume de dados:** contratos envolvem medições horárias de múltiplos pontos;
- **Sazonalidade:** variações de demanda e geração ao longo do ano;
- **Reconciliação complexa:** necessidade de conciliar dados de medição, contabilização e faturamento;
- **Penalidades:** descumprimento de obrigações pode resultar em multas significativas;
- **Lastro e garantia física:** vendedores devem comprovar capacidade de geração;
- **Múltiplos submercados:** diferenças regionais de preço exigem gestão granular.

Para de Castro et al. [2015], empresas do setor necessitam de sistemas sofisticados capazes de integrar informações da CCEE, sistemas internos de medição, contratos comerciais e sistemas financeiros.

4.5.5 Obrigações e Prazos Regulatórios

Agentes do setor elétrico devem cumprir rigorosos prazos regulatórios estabelecidos pela ANEEL e CCEE ANEEL [2022]:

- Envio mensal de dados de medição;
- Registro de contratos com antecedência mínima;
- Prestação de garantias financeiras;
- Pagamento de encargos setoriais;

- Apresentação de demonstrativos contábeis;
- Atendimento a auditorias regulatórias.

O não cumprimento desses prazos pode resultar em Câ [2023]:

- Multas regulatórias;
- Suspensão temporária de operações;
- Perda de garantias financeiras depositadas;
- Impacto na reputação corporativa;
- Dificuldades em contratações futuras.

Segundo de Souza Brito et al. [2018], a gestão eficaz de prazos é crítica e demanda sistemas que forneçam alertas automatizados e visibilidade sobre status de obrigações pendentes.

4.5.6 Transformação Digital no Setor

O setor elétrico brasileiro está passando por acelerada transformação digital, impulsionada por EPE [2020]:

- **Smart Grids:** redes inteligentes com comunicação bidirecional;
- **Medidores inteligentes:** dispositivos que transmitem dados em tempo real;
- **IoT (Internet das Coisas):** sensores distribuídos pela rede;
- **Big Data e Analytics:** análise de grandes volumes de dados operacionais;
- **Inteligência Artificial:** previsão de demanda e manutenção preditiva;
- **Blockchain:** registro descentralizado de transações energéticas;
- **Computação em nuvem:** infraestrutura escalável e flexível.

Segundo Schwab [2016], a Quarta Revolução Industrial está transformando o setor energético, exigindo que empresas adotem tecnologias digitais para permanecerem competitivas. Para Westerman et al. [2014], transformação digital não é apenas sobre tecnologia, mas sobre reimaginar processos e modelos de negócio.

4.5.7 Integração de Sistemas no Contexto Regulatório

A complexidade regulatória do setor elétrico demanda integração sofisticada entre sistemas internos e externos de Castro et al. [2015]. Empresas precisam integrar:

- **Sistemas da CCEE:** para contabilização, liquidação e registro de contratos;
- **Sistemas de medição:** coleta de dados de consumo e geração;
- **ERP corporativo:** gestão financeira, contábil e administrativa;
- **Sistemas de trading:** negociação e gestão de contratos;
- **Sistemas de CRM:** relacionamento com clientes;
- **Business Intelligence:** análise e reporting gerencial.

Segundo Tolmasquim [2016], a falta de integração adequada resulta em processos manuais intensivos, retrabalho significativo e risco elevado de não conformidade regulatória. A unificação de sistemas, portanto, não é apenas desejável mas necessária para operação eficiente e conforme no setor elétrico brasileiro.

4.5.8 Casos de Sucesso e Benchmarks

Diversas empresas do setor elétrico brasileiro e internacional têm implementado projetos de unificação e modernização de sistemas com resultados positivos EPE [2020]:

- **Redução de até 40%** no tempo de processamento de obrigações regulatórias;
- **Diminuição de 60%** em erros de entrada de dados;
- **Melhoria de 50%** na acurácia de previsões de demanda;
- **Redução de 30%** em custos operacionais de TI;
- **Aumento de 70%** na satisfação dos usuários internos.

Esses resultados reforçam a importância estratégica da unificação tecnológica para empresas do setor, conforme argumentam Laudon and Laudon [2020] e Davenport [2018].

Capítulo 5

Conclusão

Referências

Ruth Sara Aguilar-Savén. Business process modelling: Review and framework. *International Journal of Production Economics*, 90(2):129–149, 2004.

Gustavo Alonso, Fabio Casati, Harumi Kuno, and Vijay Machiraju. *Web Services: Concepts, Architectures and Applications*. Springer, Berlin, 2004.

Scott W. Ambler. *Agile Database Techniques: Effective Strategies for the Agile Software Developer*. Wiley, Indianapolis, 2003.

ANEEL. Relatório anual da agência nacional de energia elétrica. Technical report, Agência Nacional de Energia Elétrica, Brasília, 2022.

Keith H. Bennett. Legacy systems: Coping with success. *IEEE Software*, 12(1):19–23, 1995.

Gavin Bierman, Martín Abadi, and Mads Torgersen. Understanding typescript. *Proceedings of the European Conference on Object-Oriented Programming*, pages 257–281, 2014.

Connie Brooke and Rudy Ramage. Organisational scenarios and legacy systems. *International Journal of Information Management*, 18(1):3–15, 1998.

David A. Chappell. *Enterprise Service Bus*. O'Reilly Media, Sebastopol, 2004.

Regras de Comercialização: Apuração e Contabilização. Câmara de Comercialização de Energia Elétrica, São Paulo, 2023.

Thomas H. Davenport. *Process Innovation: Reengineering Work through Information Technology*. Harvard Business School Press, Boston, 1993.

Thomas H. Davenport. *The AI Advantage: How to Put the Artificial Intelligence Revolution to Work*. MIT Press, Cambridge, 2018.

Nivalde José de Castro, Rubens Rosental, and Ricardo Cavaliere. *A Retomada do Planejamento do Setor Elétrico Brasileiro*. Synergia, Rio de Janeiro, 2015.

Marcelo de Souza Brito et al. Expansão do mercado livre de energia elétrica no brasil: Análise e perspectivas. *Revista Brasileira de Energia*, 24(2):45–67, 2018.

Bonnie Eisenman. *Learning React Native: Building Native Mobile Apps with JavaScript*. O'Reilly Media, Sebastopol, 2015.

Ramez Elmasri and Shamkant B. Navathe. *Fundamentals of Database Systems*. Pearson, Boston, 7 edition, 2016.

EPE. Plano decenal de expansão de energia 2030. Technical report, Empresa de Pesquisa Energética, Rio de Janeiro, 2020.

Facebook. React - a javascript library for building user interfaces. `https://react.dev`, 2023a. Acesso em: 07 dez. 2024.

Facebook. React native - learn once, write anywhere. `https://reactnative.dev`, 2023b. Acesso em: 07 dez. 2024.

David F. Ferraiolo, Ravi Sandhu, Serban Gavrila, D. Richard Kuhn, and Ramaswamy Chandramouli. Proposed nist standard for role-based access control. *ACM Transactions on Information and System Security*, 4(3):224–274, 2001.

Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, Boston, 2002.

Dimitrios Georgakopoulos, Mark Hornick, and Amit Sheth. An overview of workflow management: From process modeling to workflow automation infrastructure. *Distributed and Parallel Databases*, 3(2):119–153, 1995.

Antonio Carlos Gil. *Métodos e Técnicas de Pesquisa Social*. Atlas, São Paulo, 7 edition, 2019.

Michael Hammer. *The Agenda: What Every Business Must Do to Dominate the Decade*. Crown Business, New York, 2001.

Michael Hammer and James Champy. Reengineering the corporation: A manifesto for business revolution. *Business Horizons*, 36(5):90–91, 1993.

Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, Boston, 2003.

William H. Inmon. *Building the Data Warehouse*. Wiley, Indianapolis, 4 edition, 2005.

Jared Hanson. Passport.js - simple, unobtrusive authentication for node.js. `http://www.passportjs.org`, 2023. Acesso em: 07 dez. 2024.

Michael B. Jones, John Bradley, and Nat Sakimura. Json web token (jwt). Technical report, May 2015.

Robert S. Kaplan and David P. Norton. *The Balanced Scorecard: Translating Strategy into Action*. Harvard Business School Press, Boston, 1996.

John P. Kotter. *Leading Change*. Harvard Business School Press, Boston, 1996.

Mary C. Lacity and Leslie P. Willcocks. Robotic process automation at telefónica o2. *MIS Quarterly Executive*, 15(1):21–35, 2016.

Kenneth C. Laudon and Jane P. Laudon. *Management Information Systems: Managing the Digital Firm*. Pearson, New York, 16 edition, 2020.

David S. Linthicum. *Enterprise Application Integration*. Addison-Wesley, Boston, 2000.

M. Lynne Markus and Cornelis Tanis. The enterprise systems experience—from adoption to success. *Framing the Domains of IT Research: Glimpsing the Future Through the Past*, 173: 173–207, 2000.

Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, Upper Saddle River, 2008.

McKinsey & Company. The social economy: Unlocking value and productivity through social technologies. Technical report, McKinsey Global Institute, 2018.

Microsoft. Typescript documentation. `https://www.typescriptlang.org`, 2020. Acesso em: 07 dez. 2024.

Andy Neely, Chris Adams, and Mike Kennerley. *The Performance Prism: The Scorecard for Measuring and Managing Business Success*. Financial Times Prentice Hall, London, 2005.

NestJS. Nestjs - a progressive node.js framework. `https://nestjs.com`, 2023. Acesso em: 07 dez. 2024.

Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Sebastopol, 2015.

Jakob Nielsen. *Usability Engineering*. Academic Press, Boston, 1993.

Donald A. Norman. *The Design of Everyday Things*. Basic Books, New York, revised and expanded edition, 2013.

Regina Obe and Leo Hsu. *PostgreSQL: Up and Running*. O'Reilly Media, Sebastopol, 3 edition, 2017.

James A. O'Brien and George M. Marakas. *Management Information Systems*. McGraw-Hill, New York, 10 edition, 2013.

PostgreSQL Global Development Group. Postgresql: The world's most advanced open source database. `https://www.postgresql.org`, 2023. Acesso em: 07 dez. 2024.

Daniel J. Power. *Decision Support Systems: Concepts and Resources for Managers*. Quorum Books, Westport, 2002.

Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, New York, 8 edition, 2016.

Niels Provos and David Mazières. A future-adaptable password scheme. *Proceedings of the USENIX Annual Technical Conference*, pages 81–91, 1999.

React Navigation. React navigation documentation. `https://reactnavigation.org`, 2023. Acesso em: 07 dez. 2024.

Denis Alcides Rezende. *Sistemas de Informações Organizacionais*. Atlas, São Paulo, 6 edition, 2017.

Jeanne W. Ross, Peter Weill, and David C. Robertson. *Enterprise Architecture as Strategy: Creating a Foundation for Business Execution*. Harvard Business School Press, Boston, 2006.

William A. Ruh, Francis X. Maginnis, and William J. Brown. *Enterprise Application Integration: A Wiley Tech Brief*. Wiley, New York, 2000.

Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, 4 edition, 2020.

Klaus Schwab. *The Fourth Industrial Revolution*. World Economic Forum, Geneva, 2016.

Ramesh Sharda, Dursun Delen, and Efraim Turban. *Business Intelligence and Analytics: Systems for Decision Support*. Pearson, Boston, 10 edition, 2014.

Alec Sharp and Patrick McDermott. *Workflow Modeling: Tools for Process Improvement and Application Development*. Artech House, Boston, 2 edition, 2009.

Mikko Siponen and Anthony Vance. Neutralization: New insights into the problem of employee information systems security policy violations. *MIS Quarterly*, 34(3):487–502, 2010.

Ralph M. Stair and George W. Reynolds. *Principles of Information Systems*. Cengage Learning, Boston, 13 edition, 2018.

Mauricio T. Tolmasquim. *Novo Modelo do Setor Elétrico Brasileiro*. Synergia, Rio de Janeiro, 2 edition, 2016.

Efraim Turban and Linda Volonino. *Information Technology for Management: Transforming Organizations in the Digital Economy*. Wiley, New Jersey, 7 edition, 2010.

TypeORM. Typeorm - orm for typescript and javascript. `https://typeorm.io`, 2023. Acesso em: 07 dez. 2024.

Wil M. P. van der Aalst and Kees Max van Hee. *Workflow Management: Models, Methods, and Systems*. MIT Press, Cambridge, 2003.

Alvaro Videla and Jason J. W. Williams. *RabbitMQ in Action: Distributed Messaging for Everyone*. Manning Publications, Greenwich, 2011.

Jan vom Brocke and Michael Rosemann. *Handbook on Business Process Management 1: Introduction, Methods, and Information Systems*. Springer, Berlin, 2007.

Jan vom Brocke and Michael Rosemann. *Handbook on Business Process Management 2: Strategic Alignment, Governance, People and Culture*. Springer, Berlin, 2 edition, 2013.

W3C. Cross-origin resource sharing (cors). `https://www.w3.org/TR/cors/`, 2014. Acesso em: 07 dez. 2024.

George Westerman, Didier Bonnet, and Andrew McAfee. *Leading Digital: Turning Technology into Business Transformation*. Harvard Business Review Press, Boston, 2014.

Stephen A. White. Introduction to bpmn. *IBM Cooperation*, 2(0):0, 2004.

Michael E. Whitman and Herbert J. Mattord. *Principles of Information Security*. Cengage Learning, Boston, 6 edition, 2018.

James P. Womack and Daniel T. Jones. *Lean Thinking: Banish Waste and Create Wealth in Your Corporation*. Free Press, New York, 2 edition, 2003.

Robert K. Yin. *Estudo de Caso: Planejamento e Métodos*. Bookman, Porto Alegre, 5 edition, 2015.